2026



PRODUCTO FUNCIONAL Y REPOSITORIO REPRODUCIBLE

“PIRISCOPE”

Sistema de estimación diaria de riesgo de incendio forestal por municipio

C088 · Proyecto de Inteligencia Artificial y Big Data · Actividad NF3 · Grupo 6
Curso de especialización en Inteligencia Artificial y Big Data · Semestre 2609

Dato	Valor
Proyecto	Piriscope — estimación diaria (D+1) del riesgo de incendio forestal en los 313 municipios de Galicia
Integrantes	Cristina Puértolas Rebollar · Hikari Lheku Barrio Martín · Guillermo Ramón Serrano · Alejandro Escolar Espejo
Fecha	7 de noviembre de 2026
Repositorio	https://github.com/cristinapuertolas-80/iFP-Piriscope
Tag de entrega	`nf3-final`
Commit evaluable	`5e20acbbaf3706ec17a7aa8fc8f1f22de06abbdd`
Evidencias del commit	Ejecución 42391224431 de GitHub Actions · rama `evidencias/nf3`

Índice

1. Datos de la entrega 3
2. Estado actual del producto 3
  2.1. Funcionalidades completas 3
  2.2. Funcionalidades parciales 4
  2.3. Pendiente para NF4 4
3. Guía de ejecución local 5
  3.1. Requisitos previos 5
  3.2. Instalación 5
  3.3. Variables de entorno 5
  3.4. Orden de arranque 6
  3.5. Comprobaciones 6
  3.6. Problemas frecuentes 7
4. Evidencias funcionales 7
  4.1. Cómo se han obtenido 7
  4.2. Backend activo 8
  4.3. Front funcionando 9
  4.4. Flujo completo y reproducibilidad 15
  4.5. Pruebas automatizadas 15
5. Arquitectura real implementada 16
  5.1. Diseño (NF2) frente a implementación 17
6. Índice de código (estructura del proyecto) 17
7. Repositorio versionado y reproducible 20
  7.1. Licencias y privacidad de los datos 20
8. Explicación por módulo 20
  8.1. Fragmentos de código relevantes 22
9. Limitaciones actuales 24
  9.1. Bugs 24
  9.2. Atajos 24
  9.3. Partes incompletas 24
  9.4. Limitaciones del modelo 25
10. Lista de comprobación de reproducibilidad 25
11. Condiciones de la entrega 25
Anexo A. Contribuciones individuales 26

Nota de transparencia: en la elaboración de este documento y en la preparación del repositorio para esta entrega se ha utilizado asistencia de herramientas de inteligencia artificial generativa (revisión del código, ejecución de comprobaciones y redacción técnica). Todas las cifras proceden de las salidas del repositorio y de la ejecución automática enlazada. El contenido ha sido revisado y validado por el equipo, que asume su autoría.

1. Datos de la entrega

Esta entrega corresponde a la fase NF3 (*Producto funcional y repositorio reproducible*). La referencia oficial de corrección es el **tag** y el **commit** indicados abajo; este documento describe exactamente ese estado del repositorio.

Campo	Valor
Nombre del proyecto	Piriscope, sistema de estimación diaria de riesgo de incendio forestal por municipio (Grupo 6, módulo C088)
Integrantes y rol principal	Cristina Puértolas Rebollar — fuentes de datos, ingesta y coordinación (Management) Hikari Lheku Barrio Martín — Machine Learning y explicabilidad (ML) Guillermo Ramón Serrano — limpieza, tipado y cruce espacio-temporal (Data Engineering) Alejandro Escolar Espejo — dashboard Streamlit (Frontend)
Fecha	7 de noviembre de 2026
Repositorio	https://github.com/cristinapuertolas-80/iFP-Piriscope (público)
Tag / commit evaluable	`nf3-final` → `5e20acbbaf3706ec17a7aa8fc8f1f22de06abbdd` (02/11/2026 07:20 CET)
Pull Requests de la entrega	#44 · producto reproducible, correcciones y evidencias y #47 · alineación tras #45 y #46
Datos y modelos	Dataset de Hugging Face cpuertolas/piriscope (acceso restringido): `raw/` (originales de cada ingesta), `data/silver/`, `data/gold/`, `models/`. Revisión usada en las evidencias: `73e7bad7bacf`
Evidencias automáticas	GitHub Actions, ejecución 42391224431 sobre el commit evaluable; ficheros en la rama `evidencias/nf3`

2. Estado actual del producto

Resumen: el producto funciona de extremo a extremo en local. En una máquina limpia, con solo un token de lectura de Hugging Face, se regenera la capa Silver a partir de los originales archivados por la ingesta, se construye Gold, se descarga el modelo v1 versionado, se generan las predicciones D+1 con sus explicaciones SHAP, se verifican automáticamente y se consultan en el dashboard. La cadena completa tardó unos 10 minutos (de los originales archivados al dashboard respondiendo; 8,5 de ellos en la descarga, porque Hugging Face limita el ritmo de peticiones) en GitHub Actions (apartado 4).

2.1. Funcionalidades completas

Funcionalidad	Dónde	Evidencia
Ingesta de AEMET, EGIF-MITECO, EFFIS/Copernicus e INE/CNIG en Bronze, con manifiesto, reintentos y archivo en Hugging Face	`src/ingest`	Workflow `ingesta.yml` (cron diario AEMET, semanal FWI) en verde; 5.593 ficheros en `raw/`
Capa Silver: limpieza, tipado, asignación de estación AEMET, código INE de cada parte y FWI por municipio, con informes de calidad y validación del modelo de datos	`src/silver`	Regenerada en las evidencias: 96 comprobaciones del modelo de datos, 0 errores, 2 avisos
Capa Gold: variables derivadas sin fuga temporal, etiqueta `incendio_d1`, particiones y municipios reservados	`src/features`	2.954.094 filas municipio-día (2001-01-01 a 2026-11-03; 2.628.887 etiquetadas, 2,99 % con incendio)
Entrenamiento y evaluación frente al baseline FWI (escenarios A, B y C, walk-forward, bootstrap por	`src/ml`	`docs/resultados/` (informe final y walk-forward)

Funcionalidad	Dónde	Evidencia
bloques)		
Inferencia batch D+1 con modelo principal (con FWI) y de respaldo (sin FWI), niveles bajo/medio/alto	`src/ml/inferir.py`	324.404 predicciones (2024-01-03 a 2026-11-04) tras regenerar desde el 02/10/2026
Explicabilidad SHAP: top-5 por predicción, valor base y resto; importancia global	`src/explainability`	`explicaciones.parquet`; figuras en `evidencias/shap/`
Verificación automática de las salidas (unicidad, cobertura, coherencia de niveles, reconstrucción de la probabilidad con SHAP)	`scripts/verificar_salidas.py`	Ejecutada por `run_pipeline.py`: «Todo correcto»
Dashboard: mapa de los 313 municipios, buscador y ficha, SHAP, evolución, tabla accesible, modelo frente a FWI, guía de lectura	`src/dashboard`	Capturas del apartado 4.3
Publicación en Hugging Face de las predicciones de Gold (tras inferir y verificar, en un único commit) y de los modelos versionados	`src/ml/hf_hub.py`, `src/ml/modelos_hf.py`	Comprobaciones previas sobre los ficheros reales con `--dry-run`: 324.404 predicciones válidas para `data/gold/` y 6 ficheros de los 2 modelos `v1` (apartado 4.2)
Orquestador único parametrizable y lanzador del dashboard en un comando	`run_pipeline.py`, `run_dashboard.py`	Usados tal cual en las evidencias
Pruebas automatizadas en cada Pull Request	`tests/`, `tests.yml`	357 pasan, 1 omitida

2.2. Funcionalidades parciales

- **Actualización diaria de las predicciones:** la ingesta es automática, pero Silver, Gold, inferencia y publicación se lanzan a mano con `run_pipeline.py` (`--subir-hf` publica la Gold tras inferir y verificar). Por eso la última fecha publicada (22/10/2026) va por detrás de la ingesta. La automatización completa era opcional en NF2.
- **Últimos días sin FWI:** el FWI llega con unos 5 días de retraso; esos días se predicen con el modelo de respaldo, que no mejora al FWI y discrimina poco (entre 12 y 18 valores distintos de probabilidad por día para unos 288 municipios en las evidencias).
- **Publicación en Hugging Face (DEC-53):** `python -m src.ml.hf_hub` / `run_pipeline.py --etapas ml --subir-hf` (Gold) y `python -m src.ml.modelos_hf --subir` (modelos) están probados con un cliente simulado y sus comprobaciones previas pasan sobre los ficheros reales (`--dry-run`), pero aún no se ha hecho una subida real con ellos.
- **Reentrenamiento (HU-11):** `run_pipeline.py --ml-acciones entrenar` está implementado y probado con datos de juguete, pero no se ha verificado todavía con datos reales que reproduzca los umbrales de `v1`.
- **Niveles de riesgo:** umbrales fijados en validación por Recall objetivo (0,8 y 0,5), pendientes de confirmar (DEC-49); en episodios de sequía casi todos los municipios quedan en «medio» o «alto».
- **Ingesta de EGIF y CNIG:** requiere dejar a mano las exportaciones en `data/landing/` (las fuentes no ofrecen API).

2.3. Pendiente para NF4

- Primera publicación real con `--subir-hf` y encadenar Silver → Gold → inferencia → publicación tras la ingesta (workflow programado).
- Verificar el reentrenamiento reproducible con datos reales y reajustar los umbrales con el último año completo.
- Cerrar la política de niveles (DEC-49) y valorar la calibración de la probabilidad.
- Comparar el FWI intermedio con el consolidado y contrastar las etiquetas de 2022-2023 con las estadísticas definitivas de EGIF.

- Memoria final: documentar que el modelo no mejora al FWI en A Coruña (ya indicado en el dashboard) y presentación.

3. Guía de ejecución local

Todos los comandos se ejecutan desde la raíz del repositorio. La misma guía está en el `README.md` (apartados 1 a 8 y 6 bis) y es la que ejecuta el workflow de evidencias en una máquina limpia.

3.1. Requisitos previos

Requisito	Para qué	Necesario en
Python 3.12 (probado; vale ≥ 3.10) y Git	Ejecutar el proyecto	Siempre
Cuenta de Hugging Face con acceso aceptado a `cpuertolas/piriscope` y token de lectura	Leer originales archivados, Silver, Gold y modelos (dataset de acceso restringido)	Opciones A y B
4 GB de RAM o más	Silver y Gold trabajan con ~2,9 millones de filas (memoria máxima medida: 2,7 GB en Gold)	Opción B
API key de AEMET OpenData y clave del Early Warning Data Store (Copernicus)	Descargar datos nuevos de las fuentes	Solo opción C
Token de Hugging Face con escritura sobre el dataset	Publicar originales, Silver, Gold y modelos	Solo para publicar

3.2. Instalación

```
git clone https://github.com/cristinapuertolas-80/iFP-Piriscope.git
cd iFP-Piriscope
git checkout nf3-final                    # versión evaluable
python -m venv .venv
source .venv/bin/activate                # Windows: .venv\Scripts\activate
pip install -r environment/requirements-lock.txt
cp .env.example .env                      # y rellenar HF_TOKEN (y las claves si se usa la opción C)
```

`environment/requirements-lock.txt` fija las versiones exactas con las que pasan los tests y se generaron las evidencias (Python 3.12; válido para Linux, Windows y macOS). `requirements.txt` declara los rangos mínimos y `requirements-dashboard.txt` solo lo que necesita el dashboard.

3.3. Variables de entorno

Se leen del fichero `.env` de la raíz (excluido en `.gitignore`); `.env.example` es la plantilla sin secretos.

Variable	Obligatoria	Descripción
`HF_TOKEN`	Sí (A y B)	Token de Hugging Face. Lectura para ejecutar; escritura solo para publicar (`--subir-hf`, `modelos_hf --subir`)
`HF_REPO_ID`	No	Dataset de datos y modelos (por defecto `cpuertolas/piriscope`)
`AEMET_API_KEY`	Solo C	API key de AEMET OpenData
`CDSAPI_URL`, `CDSAPI_KEY`	Solo C (FWI)	Early Warning Data Store de Copernicus
`PIRISCOPE_DATA_DIR`	No	Carpeta de datos (por defecto `./data`)
`PIRISCOPE_DASHBOARD_ORIGEN`	No	`local` fuerza al dashboard a leer `data/gold/` (lo activa `run_dashboard.py --local`)

Variable	Obligatoria	Descripción
`AEMET_MAX_DIAS`, `AEMET_PAUSA_SEGUNDOS`, `EFFIS_*`…	No	Ajustes de la ingesta, documentados en `.env.example`

3.4. Orden de arranque

El sistema es batch: primero el pipeline genera Gold (predicciones y explicaciones) y después el dashboard la lee. No hay servicios que arrancar en paralelo ni base de datos.

Opción A · Solo consultar el dashboard (1 comando)

```
# Crea .venv si hace falta, pide el token, comprueba el acceso y abre http://localhost:8501
python run_dashboard.py
```

Lee la Gold publicada en Hugging Face. Opciones: `--local` (lee `data/gold/`), `--cambiar-token`, `--puerto`, `--sin-navegador`, `--solo-preparar`.

Opción B · Levantar todo el producto desde cero con los datos archivados (recomendada para corregir)

```
# 1. Originales archivados en raw/ -> Silver (4-9 min: Hugging Face limita las descargas)
python -m src.silver --descargar-hf
# 2. Silver -> Gold (~10 s)
python run_pipeline.py --etapas gold
# 3. Modelo v1 de Hugging Face -> predicciones D+1, SHAP y verificación automática (~15 s)
python run_pipeline.py --etapas ml --descargar-modelos --inferir-desde 2026-10-01
# 4. Dashboard sobre la Gold recién generada (http://localhost:8501)
python run_dashboard.py --local
```

El paso 3 descarga `models/v1_*` del dataset (`python -m src.ml.modelos_hf`), predice desde la fecha indicada hasta el último día con datos y ejecuta `scripts/verificar_salidas.py`; si una comprobación falla, el comando termina con código 1. Sin `--inferir-desde` predice solo el último día disponible. Duraciones medidas en un runner estándar de GitHub Actions.

Opción C · Incorporar datos nuevos de las fuentes

```
python run_pipeline.py --desde 2026-10-20 --hasta 2026-10-31 --etapas ingesta silver --fuentes aemet
effis
python run_pipeline.py --etapas gold ml --subir-hf      # predice, verifica y publica la Gold en Hugging
Face
# Reentrenar con un periodo nuevo (HU-11): modelo versionado nuevo, sin editar código
python run_pipeline.py --etapas gold ml --ml-acciones entrenar inferir --version-modelo v2
```

Necesita `AEMET_API_KEY` y `CDSAPI_KEY`. Se ejecuta después de la opción B (o de la carga inicial de `scripts/carga_inicial.sh` / `.ps1`): Silver combina las ingestas nuevas de `data/bronze/` con los originales ya descargados en `data/hf/raw/`. Esta opción no forma parte de las evidencias automáticas porque requiere las claves de las fuentes; la ingesta sí se ejecuta a diario con `ingesta.yml` (última ejecución archivada de AEMET: `raw/AEMET/20261107T110638Z`). Si falla Silver, Gold o ML el pipeline se detiene (`--continuar` lo evita).

3.5. Comprobaciones

```
pytest                                                    # 358 tests, sin red ni datos reales
python scripts/verificar_salidas.py --pred data/gold/predicciones.parquet --expl
data/gold/explicaciones.parquet
python scripts/comparar_predicciones.py --referencia <predicciones publicadas> --nuevo
data/gold/predicciones.parquet
```

```
curl http://localhost:8501/_stcore/health                    # el servidor del dashboard responde «ok»
```

3.6. Problemas frecuentes

Síntoma	Causa y solución
`401`/`403` o «acceso denegado» al descargar	La cuenta no ha aceptado las condiciones del dataset o el token no es de lectura. `python run_dashboard.py --cambiar-token`
«No hay modelos completos de la versión v1»	Falta `HF_TOKEN` en `.env` o no hay acceso al dataset
El mapa sale sin fondo	Las teselas de CARTO/OpenStreetMap necesitan internet; los municipios se pintan igualmente
Puerto 8501 ocupado	`run_dashboard.py` busca el siguiente libre o se indica con `--puerto`

4. Evidencias funcionales

4.1. Cómo se han obtenido

Las evidencias no son capturas manuales: las genera el workflow `.github/workflows/evidencias.yml` del propio repositorio sobre el **commit evaluable** (`5e20acb`), en una máquina limpia de GitHub Actions (Python 3.12.16, Linux x86_64), siguiendo la opción B de la guía. Cualquiera con acceso al dataset puede repetirlas lanzando el workflow. La ejecución es la 42391224431 (07/11/2026 16:25:35 → 16:37:06 UTC) y sus ficheros están en la rama `evidencias/nf3`.

Paso	Comando	Resultado	Tiempo
Entorno	`pip install -r environment/requirements-lock.txt`	Python 3.12.16 con las versiones del lockfile	36 s
Tests	`pytest`	357 pasan, 1 omitida (necesita Silver real)	32 s
Bronze → Silver	`python -m src.silver --descargar-hf`	Descarga `raw/` (AEMET, EFFIS, EGIF, INE_CNIG); 5 tablas; validación 96 / 0 errores / 2 avisos	8 min 31 s
Silver → Gold	`python run_pipeline.py --etapas gold`	2.954.094 filas municipio-día, 2001-01-01 a 2026-11-03	9 s
Gold → ML	`python run_pipeline.py --etapas ml --descargar-modelos --inferir-desde 2026-10-01`	Modelos v1 descargados; predicciones hasta 2026-11-04 con SHAP; verificación «Todo correcto»	13 s
Publicación (simulada)	`python -m src.ml.hf_hub --dry-run` y `python -m src.ml.modelos_hf --subir --dry-run`	Gold y modelos pasan las comprobaciones previas; no se sube nada	3 s
Reproducibilidad	`python scripts/comparar_predicciones.py …`	Las 5.947 predicciones comunes coinciden con las publicadas	1 s
Dashboard	`python run_dashboard.py --local --sin-navegador`	Crea `.venv`, instala sus librerías y arranca; `/_stcore/health` → ok, `GET /` → 200	24 s
Capturas	`python scripts/capturar_evidencias.py`	Figuras 1 a 7	45 s

4.2. Backend activo

En Piriscope el backend es el proceso batch que produce la capa Gold (NF2, apartado 5.6) y el servidor de Streamlit que la sirve. Extractos de los registros de la ejecución (sin las marcas de tiempo y con las rutas abreviadas):

evidencias/04_silver.log (extracto)

```
piriscope.silver.hf | Descargando raw/EGIF de Hugging Face…
WARNING huggingface_hub | HTTP Error 429 … Rate limited. Waiting 57.0s before retry [Retry 1/5].
(varias veces)
piriscope.silver | Silver municipio: 313 filas
piriscope.silver | Silver estacion_aemet: 58 filas
piriscope.silver.meteo | AEMET: 455303 registros leídos, 442576 tras quitar repetidos entre ingestas
piriscope.silver | Silver observacion_meteorologica: 2954094 filas
piriscope.silver.fwi | FWI: 304 ficheros (7 intermedios)
piriscope.silver | Silver indicador_fwi: 2866454 filas
piriscope.silver | Silver evento_incendio: 264681 filas
WARNING piriscope.silver.validacion | [observacion_meteorologica] cobertura_viento: 56.1 % nulos
WARNING piriscope.silver.validacion | [indicador_fwi] dias_sin_fwi: 273 días sin FWI (meses: ['2002-
01', ...])
piriscope.silver | Validación: 96 comprobaciones, 0 errores, 2 avisos (informes/validacion.json)
duración: 8:31.16 · memoria máx: 1883416 KB
```

evidencias/05_gold.log (extracto)

```
piriscope.pipeline | === Gold: python -m src.features ===
Filas: 2,954,094 (2,628,887 etiquetadas) · 2001-01-01 a 2026-11-03
Días con incendio (positivos): 78,706 = 2.994 % de las filas etiquetadas
Prevalencia % por partición: {'entrenamiento': 3.178, 'test': 0.75, 'validacion': 1.366}
duración: 0:08.83 · memoria máx: 2693712 KB
```

evidencias/06_ml.log (extracto): inferencia y verificación automática

```
piriscope.ml.modelos_hf | Modelos v1 descargados en .../models: v1_xgboost_con_fwi-sin_viento-desde2009,
                          v1_xgboost_sin_fwi-sin_viento-desde2009
Modelo v1_xgboost_con_fwi-sin_viento-desde2009 · 10,465 predicciones para 2026-10-02 a 2026-11-04 (313
municipios)
Niveles de riesgo en la última fecha: {'bajo': 221, 'medio': 67, 'alto': 0}
Cobertura en la última fecha: 288 de 313 municipios
Fuera del dominio del modelo principal (falta alguna variable que nunca faltó al entrenar): 2,191 filas
  -> 2,014 con el modelo de respaldo, 177 sin predicción
Sustituidas 6,498 predicciones anteriores del mismo municipio y fecha (p. ej. v1-respaldo por v1)
324,404 predicciones · 2024-01-03 a 2026-11-04 · versiones: {'v1': 322390, 'v1-respaldo': 2014}
[OK] id_prediccion único
[OK] una sola predicción por municipio y fecha
[OK] probabilidades en [0, 1] y sin nulos
[OK] explicaciones y predicciones con los mismos ids · 0 ids sin pareja
[OK] 5 variables por predicción · {5: 324404}
[OK] valor_base + SHAP reconstruye la probabilidad · diferencia máxima 1.19e-07
[OK] las filas de respaldo no llevan FWI
[OK] cobertura >= 80% en todas las fechas · mínima 91% el 2026-10-29; 7 fecha(s) incompletas
[OK] niveles coherentes con los umbrales del modelo principal · medio=0.0273, alto=0.0449
[OK] niveles coherentes con los umbrales del modelo respaldo · medio=0.0220, alto=0.0241
[AVISO] 84 fecha(s) con >= 95% en medio o alto, solo con modelo principal (suele ser sequía real, ver
DEC-49)
Todo correcto
duración: 0:13.50 · memoria máx: 1933136 KB
```

evidencias/06b_publicacion_dry_run.txt: comprobaciones de publicación sobre los ficheros reales, sin subir nada

```
### python -m src.ml.hf_hub --dry-run
INFO Gold a publicar en cpuertolas/piriscope (dataset/data/gold): 324.404 predicciones de 2024-01-03 a
2026-11-04
INFO Simulación: no se sube nada. Token presente.

### python -m src.ml.modelos_hf --subir --version v1 --dry-run
piriscope.ml.modelos_hf | Modelos a publicar en cpuertolas/piriscope (dataset/models):
  v1_xgboost_con_fwi-sin_viento-desde2009.joblib, v1_xgboost_con_fwi-sin_viento-desde2009.json,
  v1_xgboost_con_fwi-sin_viento-desde2009_umbrales.json, v1_xgboost_sin_fwi-sin_viento-desde2009.joblib,
  v1_xgboost_sin_fwi-sin_viento-desde2009.json, v1_xgboost_sin_fwi-sin_viento-desde2009_umbrales.json
piriscope.ml.modelos_hf | Simulación: no se sube nada. Token presente.
```

evidencias/08_dashboard.log y 08_health.txt: el servidor del dashboard responde

```
> 1/3 Entorno de Python
  Creando el entorno virtual en .../.venv (solo la primera vez)…
> 2/3 Librerías
  Instalando las librerías del dashboard (1-2 minutos la primera vez)…
> 3/3 Acceso a los datos
  Se usan los ficheros de .../data/gold.
2026-11-07 16:36:17.518 Uvicorn server started on :::8501
  Dashboard abierto en http://localhost:8501

2026-11-07T16:36:17Z
GET /_stcore/health -> ok
GET / -> HTTP 200
```

4.3. Front funcionando

Capturas del dashboard lanzado con `run_dashboard.py --local` sobre la Gold que acaba de generar el pipeline (la barra lateral lo indica: «Datos: ficheros locales de data/gold · Predicciones generadas el 07/11/2026 16:35 UTC»).

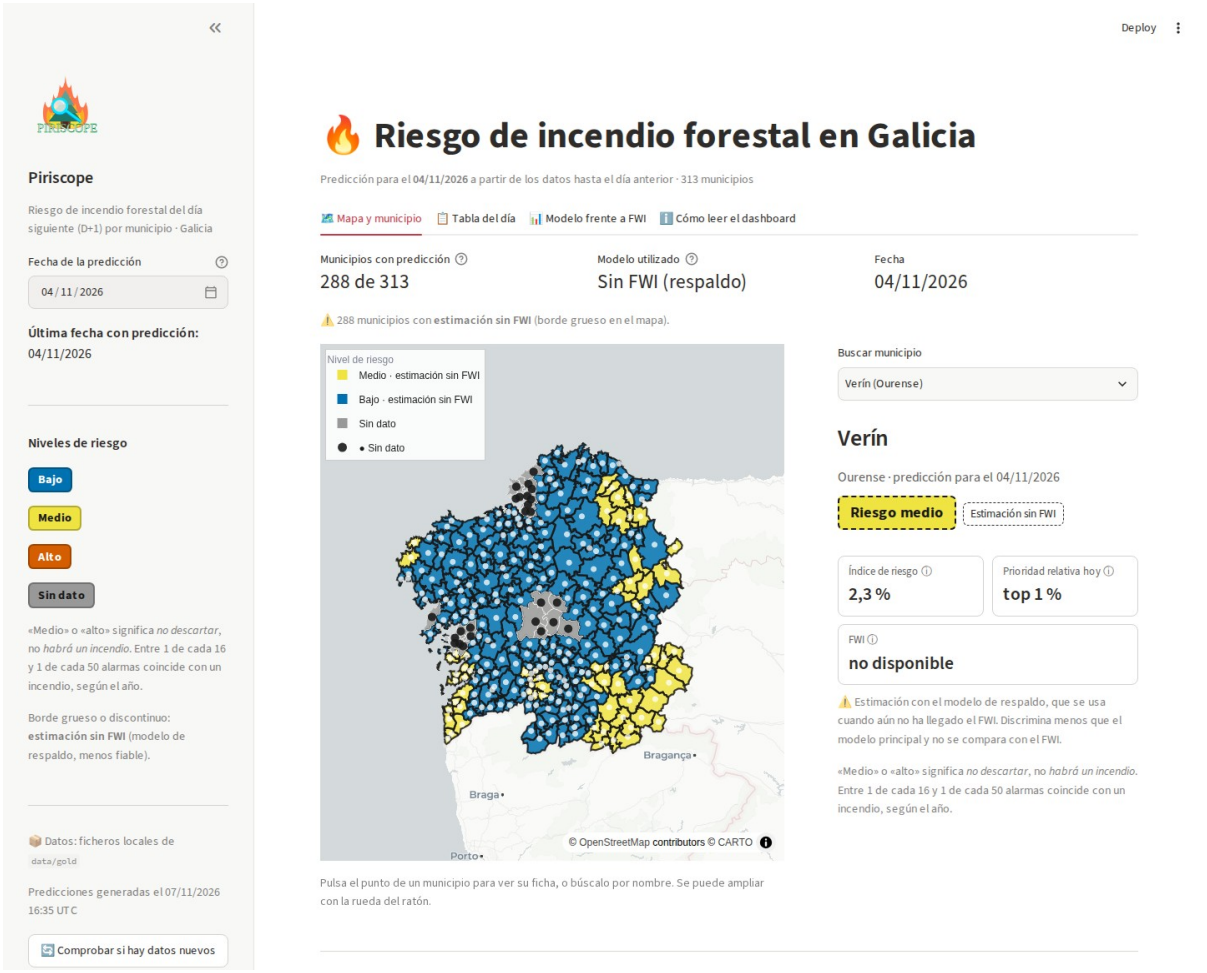


Figura 1. Vista inicial: última fecha con predicción (04/11/2026), mapa y ficha del municipio con mayor índice. Esos días aún no tienen FWI: se usa el modelo de respaldo, lo que se indica con el borde discontinuo, la etiqueta «Estimación sin FWI» y el aviso de la ficha. Los municipios sin datos meteorológicos salen como «Sin dato» (gris con punto oscuro).

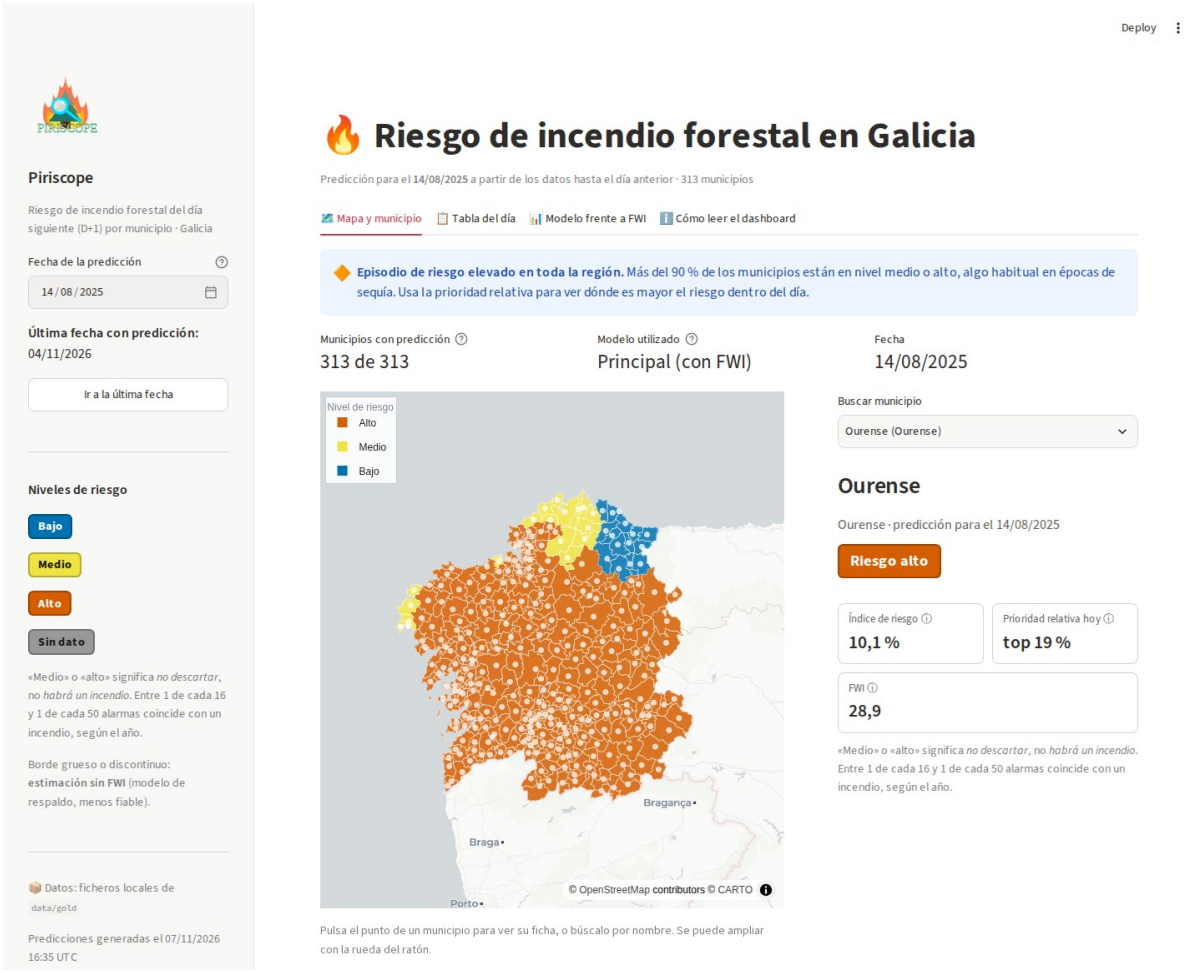


Figura 2. Consulta de un municipio y una fecha por URL (`?fecha=2025-08-14&municipio=32054`, Ourense): aviso de episodio de riesgo generalizado, modelo principal (con FWI), nivel en texto, índice de riesgo, prioridad relativa y FWI.

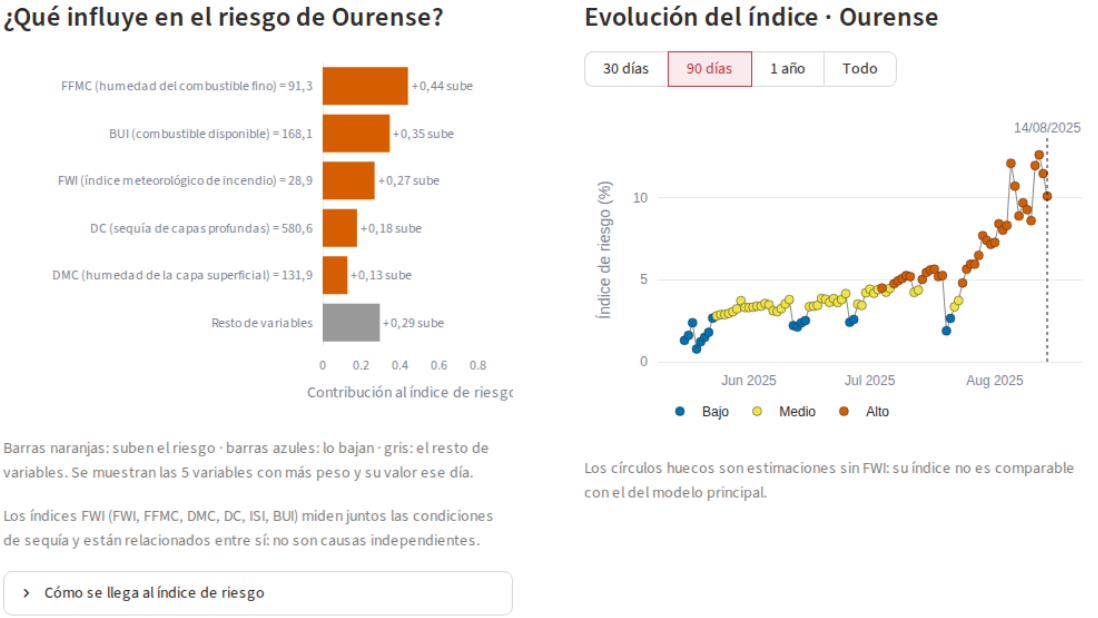
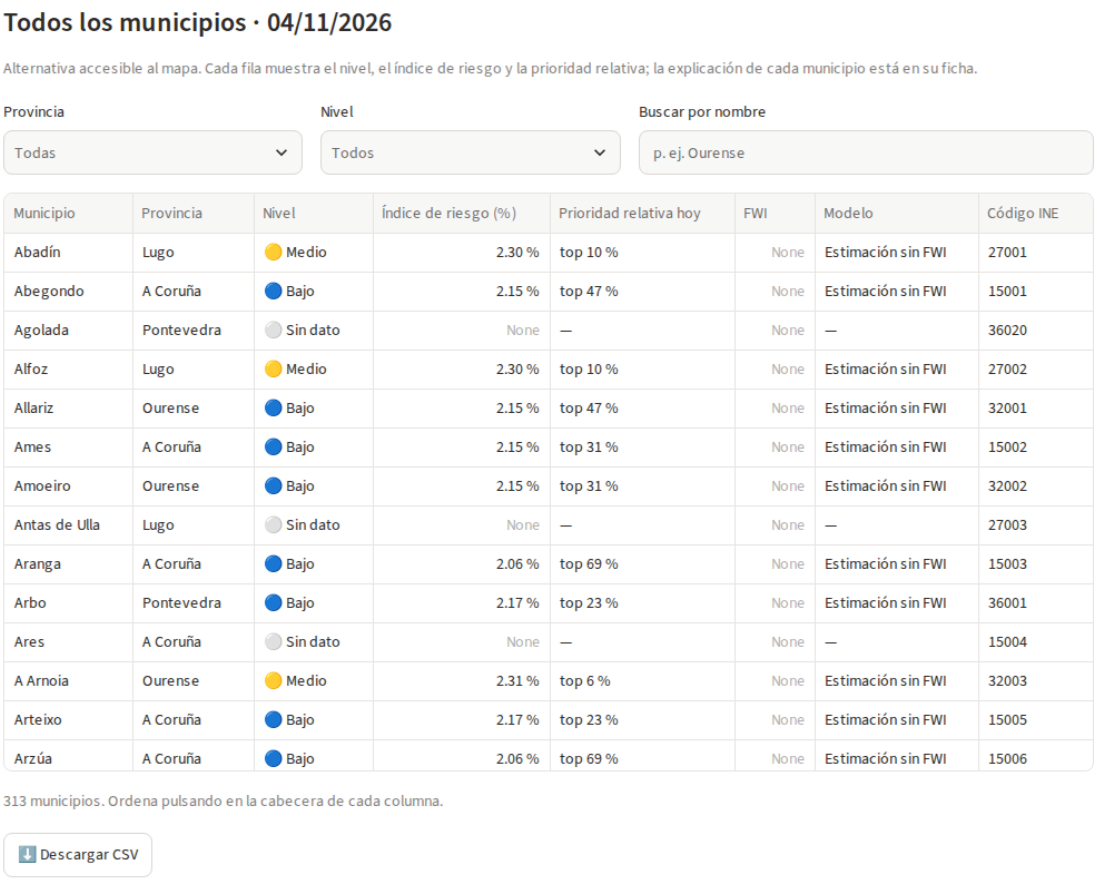


Figura 3. Continuación de la figura 2: variables que más influyen en la predicción (SHAP, con su valor ese día) y evolución del índice del municipio en los 90 días anteriores.



Todos los municipios · 04/11/2026

Alternativa accesible al mapa. Cada fila muestra el nivel, el índice de riesgo y la prioridad relativa; la explicación de cada municipio está en su ficha.

Provincia: Todas | Nivel: Todos | Buscar por nombre: p. ej. Ourense

Municipio	Provincia	Nivel	Índice de riesgo (%)	Prioridad relativa hoy	FWI	Modelo	Código INE
Abadín	Lugo	🟡 Medio	2.30 %	top 10 %	None	Estimación sin FWI	27001
Abegondo	A Coruña	🔵 Bajo	2.15 %	top 47 %	None	Estimación sin FWI	15001
Agolada	Pontevedra	⚪ Sin dato	None	—	None	—	36020
Alfoz	Lugo	🟡 Medio	2.30 %	top 10 %	None	Estimación sin FWI	27002
Allariz	Ourense	🔵 Bajo	2.15 %	top 47 %	None	Estimación sin FWI	32001
Ames	A Coruña	🔵 Bajo	2.15 %	top 31 %	None	Estimación sin FWI	15002
Amoeiro	Ourense	🔵 Bajo	2.15 %	top 31 %	None	Estimación sin FWI	32002
Antas de Ulla	Lugo	⚪ Sin dato	None	—	None	—	27003
Aranga	A Coruña	🔵 Bajo	2.06 %	top 69 %	None	Estimación sin FWI	15003
Arbo	Pontevedra	🔵 Bajo	2.17 %	top 23 %	None	Estimación sin FWI	36001
Ares	A Coruña	⚪ Sin dato	None	—	None	—	15004
A Arnoia	Ourense	🟡 Medio	2.31 %	top 6 %	None	Estimación sin FWI	32003
Arteixo	A Coruña	🔵 Bajo	2.17 %	top 23 %	None	Estimación sin FWI	15005
Arzúa	A Coruña	🔵 Bajo	2.06 %	top 69 %	None	Estimación sin FWI	15006

313 municipios. Ordena pulsando en la cabecera de cada columna.

⬇️ Descargar CSV

Figura 4. Tabla del día: alternativa accesible al mapa, con filtros por provincia, nivel y nombre, orden por columnas y descarga en CSV.



Figura 5. Modelo frente al FWI (escenario A, test 2023): AUC-PR con intervalo, veces el azar y desglose por provincia, con el aviso de que el modelo no mejora claramente en A Coruña.

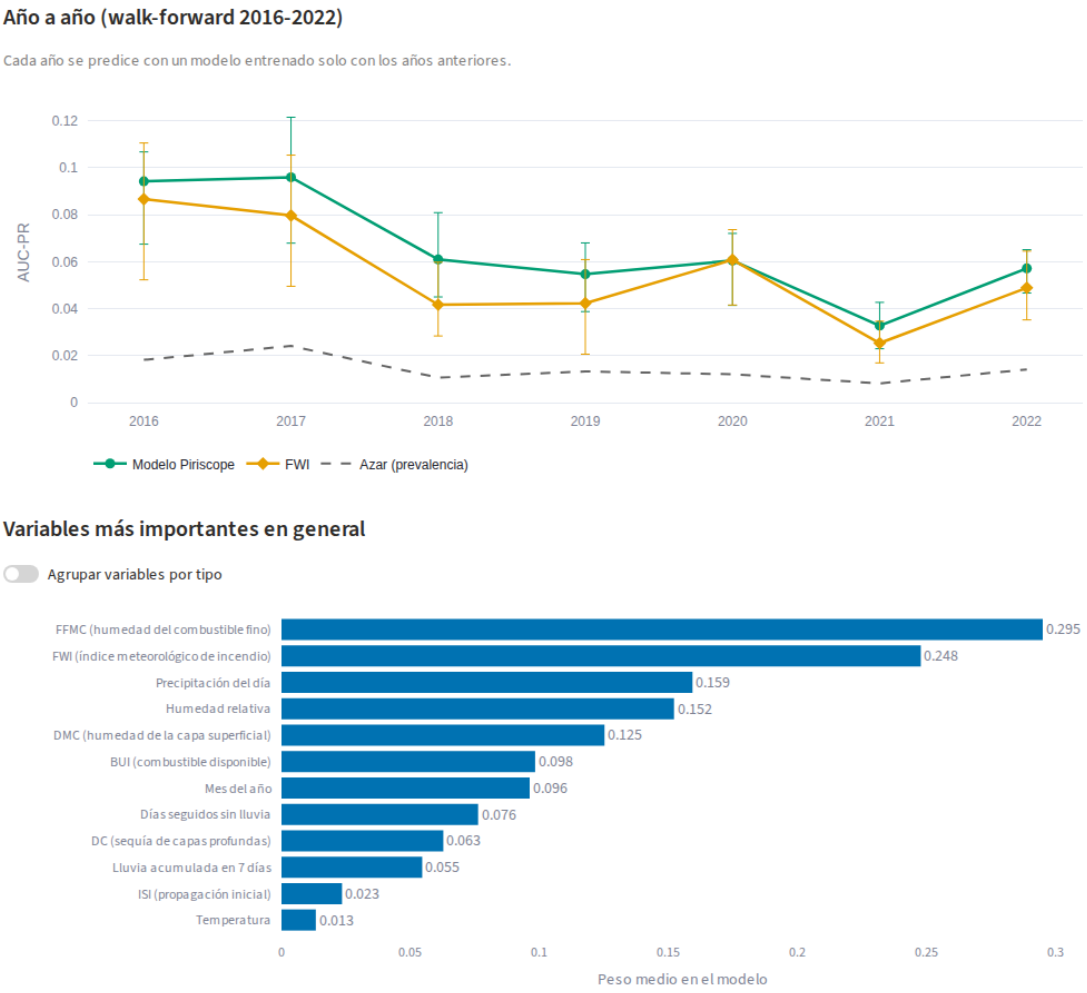


Figura 6. Validación año a año (walk-forward 2016-2022) e importancia global de las variables (SHAP).

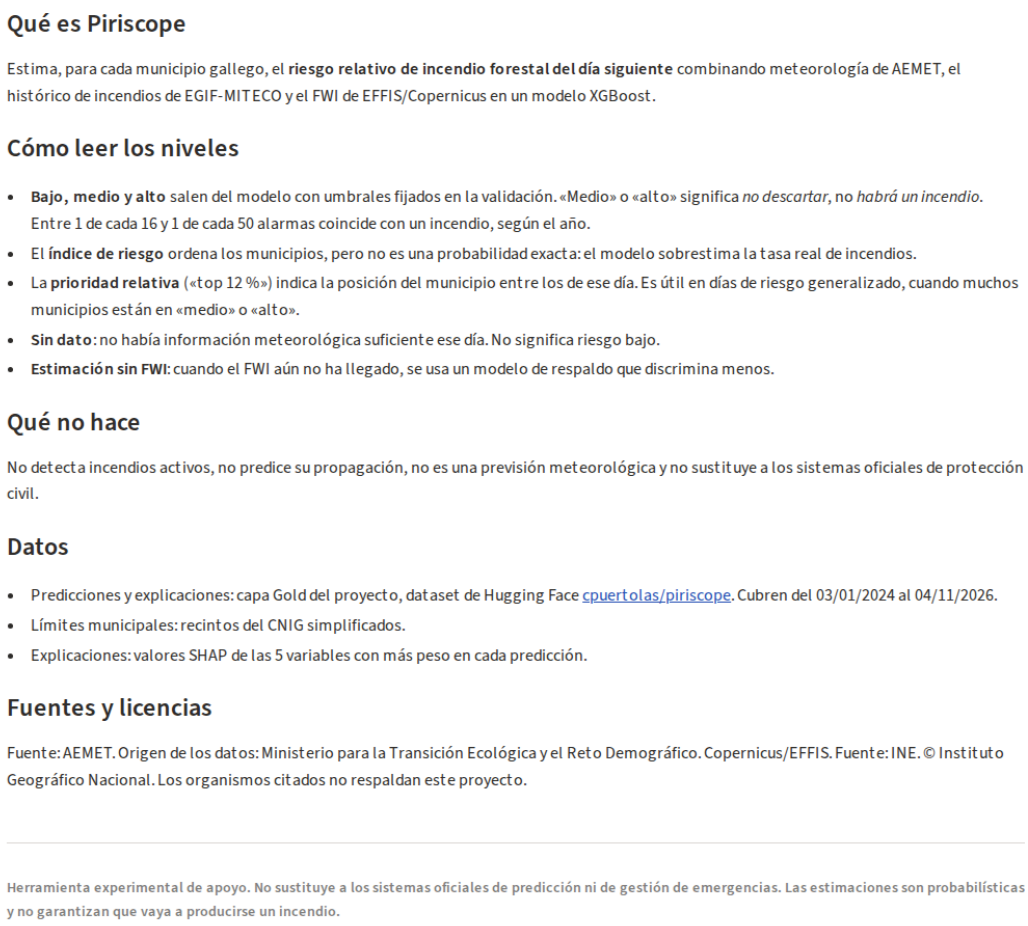


Figura 7. Cómo leer el dashboard: niveles, índice, prioridad relativa, «Sin dato», estimación sin FWI, qué no hace y fuentes.

4.4. Flujo completo y reproducibilidad

El flujo de la figura 8 (apartado 5) se ha recorrido completo: originales archivados → Silver → Gold → modelo versionado → predicciones y SHAP → verificación → dashboard. Además, las predicciones regeneradas se compararon con las publicadas en Hugging Face (`scripts/comparar_predicciones.py`):

evidencias/07_reproducibilidad.txt

```
version_modelo  en_ambos  solo_referencia  solo_nuevo  dif_max_probabilidad  dif_media_probabilidad  cambios_de_nivel
         total      5947              864        4831          1.862645e-09            4.698113e-13                 0
            v1      5947                0        2817          1.862645e-09            4.698113e-13                 0
   v1-respaldo         0              864        2014                   NaN                     NaN                 0

REPRODUCIDO: 5,947 predicciones comunes, diferencia máxima 1.86e-09 (tolerancia 1e-06), 0 cambios de nivel

Predicciones comunes con diferencia > 1e-6: 0
```

Lectura: las 5.947 predicciones comunes con las publicadas coinciden (diferencia máxima $1{,}9 \cdot 10^{-9}$, error de coma flotante) y ninguna cambia de nivel. Las 864 predicciones publicadas con el modelo de respaldo aparecen «solo en la referencia» porque esos días ya tienen FWI (llegó con la ingesta del 02/11/2026) y ahora se predicen con el modelo principal; las 4.831 «solo nuevas» son esos mismos días con el modelo principal y los días posteriores a la última publicación (hasta el 04/11/2026). Conclusión: con las mismas entradas, el pipeline reproduce las salidas publicadas.

4.5. Pruebas automatizadas

`pytest` ejecuta 358 pruebas en unos 30 s (357 pasan, 1 omitida). No necesitan red ni datos reales: usan datos de juguete con el esquema de cada capa y dobles de las APIs. Se ejecutan en cada Pull Request (`tests.yml`, con

el lockfile e informe JUnit) y en el workflow de evidencias. La prueba omitida valida las tablas Silver reales y solo se ejecuta si existen en `data/silver`; con datos reales esa validación la hace `python -m src.silver` (96 comprobaciones, 0 errores).

Área	Ficheros de test	Qué comprueban (ejemplos)
Ingesta	test_aemet, test_effis, test_egif, test_cli, test_common, test_config, test_hf_hub, test_errores_red	Troceo de peticiones, reintentos, caché, manifiestos, errores de red y SSL, subida a HF
Silver	test_silver_* (7 ficheros)	Tipado, asignación de estación e INE, FWI por celda, calidad, validación del modelo de datos
Gold	test_features	Las derivadas de D no cambian si se alteran datos posteriores a D (sin fuga temporal)
ML	test_config, test_splits, test_train, test_evaluate, test_risk, test_predict, test_reservados, test_experimento, test_walkforward, test_inferir, test_modelos_hf, test_hf_gold	Sin variables prohibidas, entrenamiento sin filas reservadas ni de test, mismas filas modelo/FWI, dominio y respaldo, publicación coherente en Hugging Face
SHAP	test_shap, test_shap_valores	Top-5, valor base + SHAP reconstruye la predicción
Dashboard	test_dashboard, test_run_dashboard	Casos límite del contrato Gold y smoke test de la app con datos de juguete
Orquestación y scripts	test_run_pipeline_ml, test_scripts	Orden y parada de etapas, verificación tras inferir, descarga de modelos, todos los scripts ejecutables

5. Arquitectura real implementada

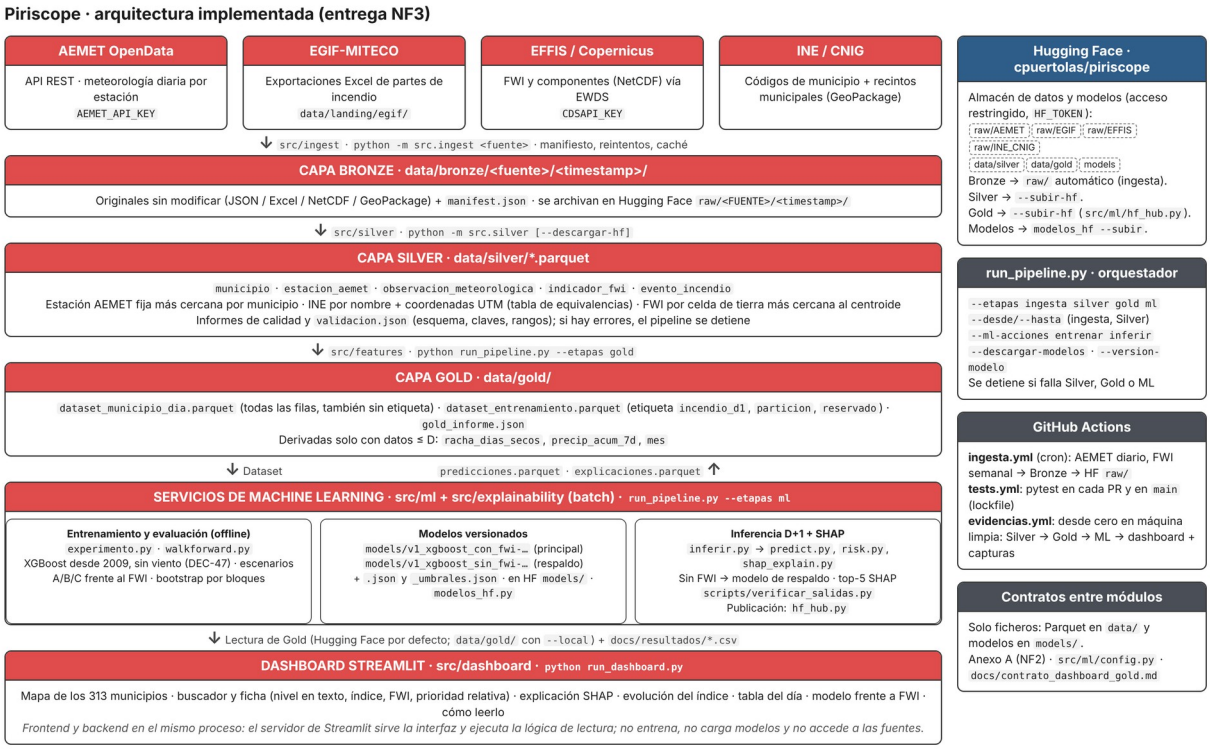


Figura 8. Arquitectura implementada en el commit evaluable (fuente editable: `docs/arquitectura_implementada.html`).

Explicación. Piriscope es un pipeline batch con arquitectura medallón sobre ficheros. `src/ingest` descarga cada fuente y guarda los originales sin modificar en `data/bronze/<fuente>/<timestamp>/` con un manifiesto, y los archiva en `raw/` del dataset de Hugging Face; el workflow `ingesta.yml` lo hace a diario (AEMET) y semanalmente (FWI). `src/silver` limpia, tipa y cruza las fuentes en cinco tablas Parquet que siguen

el modelo de datos de NF2 y valida esquema, claves y rangos. `src/features` construye Gold: una fila por municipio y día con las derivadas, la etiqueta y las particiones. `src/ml` entrena y evalúa offline frente al FWI y, en producción, `inferir.py` carga los modelos versionados, separa las filas fuera del dominio del modelo principal (sin FWI) hacia el modelo de respaldo, calcula SHAP con `src/explainability` y escribe `predicciones.parquet` y `explicaciones.parquet`; tras verificarlos, `src/ml/hf_hub.py` puede publicarlos en `data/gold/` del dataset (`--subir-hf`). El dashboard solo lee esos dos ficheros (de Hugging Face o de `data/gold/`) y los informes de `docs/resultados/`. Los módulos se comunican únicamente mediante ficheros; `run_pipeline.py` los encadena.

5.1. Diseño (NF2) frente a implementación

Elemento del diseño NF2	Implementado	¿Coincide?
Medallón Bronze/Silver/Gold en `data/` sin base de datos	Igual; Parquet en Silver y Gold	Sí
Hugging Face como almacén de datos y modelos; ingesta automática con HF_T0KEN en Secrets	`raw/`, `data/silver`, `data/gold`, `models/`; `ingesta.yml`; publicación de Gold y modelos con `hf_hub.py` y `modelos_hf.py` (DEC-53)	Sí
Cuatro fuentes: AEMET, EGIF-MITECO, EFFIS/Copernicus, INE/CNIG	Un módulo por fuente en `src/ingest`	Sí
Estación AEMET más cercana con distancia; INE por nombre validado con UTM; FWI por centroide	Estación fija más cercana (DEC-33); FWI de la celda de tierra más cercana al centroide (DEC-34)	Sí, con ajustes documentados
Derivadas: racha de días secos, precipitación acumulada…	`racha_dias_secos`, `precip_acum_7d`, `mes`, sin fuga temporal (test)	Sí
XGBoost y/o Random Forest; partición temporal y geográfica; escenarios A/B/C; Recall y AUC-PR; bootstrap	Ambos algoritmos; elegido XGBoost (DEC-47); se añade validación walk-forward y bootstrap por bloques	Sí (ampliado)
Inferencia batch D+1 en Gold y SHAP en el mismo proceso	`inferir.py`; se añade modelo de respaldo sin FWI para los días sin FWI (DEC-48)	Sí (ampliado)
Modelos serializados y versionados en `models/`	`models/v1_*` (+ metadatos y umbrales), alojados en Hugging Face (DEC-44) y descargables	Sí
Entidades Prediccion y ExplicacionPrediccion	Mismas claves; ExplicacionPrediccion añade `valor_variable`, `valor_base`, `shap_resto` (DEC-50)	Sí (ampliado)
Streamlit monolítico que solo lee Gold; mapa, buscador, fecha, FWI, comparación, SHAP, tabla	Las siete funciones; Gold desde Hugging Face por defecto (DEC-52)	Sí
`run_pipeline.py` parametrizable que encadena ingesta → Silver → Gold → ML	Etapas, rango de fechas, entrenar/inferir, versión de modelo, descarga de modelos	Sí
pytest + GitHub Actions en cada PR	`tests.yml` (lockfile, JUnit) + `evidencias.yml`	Sí (ampliado)
Opcionales: BD, PySpark, NDVI/terreno, automatización completa	No implementados	Según lo previsto

Todas las desviaciones están registradas en `docs/decisiones.md` (DEC-30 a DEC-54).

6. Índice de código (estructura del proyecto)

Estructura real del repositorio en `5e20acb` (se omiten los `__init__.py` y los `README.md` de cada carpeta). 151 ficheros versionados; 7.129 líneas de Python en `src/` y 3.870 en `tests/`.

```
iFP-Piriscope/
├── .env.example  # variables de entorno (sin secretos)
├── .gitattributes
├── .gitignore
├── CONTRIBUTING.md
```

```
├── pytest.ini  # configuración de pytest
├── README.md
├── run_dashboard.py  # lanzador del dashboard
├── run_pipeline.py  # orquestador: ingesta → Silver → Gold → ML
├── .github/
│   └── workflows/  # CI: tests, ingesta (cron), evidencias
│       ├── evidencias.yml
│       ├── ingesta.yml
│       └── tests.yml
├── docs/  # entregas, arquitectura, decisiones, resultados
│   ├── assets/
│   │   └── logo-piriscope.svg
│   ├── resultados/
│   │   ├── dev/
│   │   │   ├── informe_walkforward_repro.csv
│   │   │   ├── walkforward_predicciones.parquet
│   │   │   └── walkforward_predicciones_umbrales.csv
│   │   ├── gold_informe.json
│   │   ├── informe_final.csv
│   │   ├── informe_walkforward.csv
│   │   ├── shap_cascada_32031_2026-10-19.html
│   │   ├── shap_global.csv
│   │   ├── shap_global.html
│   │   ├── shap_global_grupos.csv
│   │   ├── shap_global_grupos.html
│   │   └── shap_local_32031_2026-10-19.html
│   ├── arquitectura.png
│   ├── arquitectura_implementada.html
│   ├── arquitectura_implementada.png
│   ├── contrato_dashboard_gold.md
│   ├── cronograma.pdf
│   ├── decisiones.md
│   ├── incendios_muestra_validacion.csv
│   ├── modeloDatos.png
│   ├── municipios_reservados.csv
│   ├── municipios_reservados_meta.json
│   ├── NF1_Presentacion_y_Viabilidad.pdf
│   ├── NF2_Documentacion_y_Diseno_Tecnico.pdf
│   ├── plan_ingesta.md
│   └── tabla_equivalencias.csv
├── environment/  # dependencias
│   ├── requirements-dashboard.txt
│   ├── requirements-lock.txt
│   └── requirements.txt
├── scripts/  # carga inicial, verificación, análisis, capturas
│   ├── analisis_niveles.py
│   ├── analisis_walkforward.py
│   ├── capturar_evidencias.py
│   ├── carga_inicial.ps1
│   ├── carga_inicial.sh
│   ├── comparar_fwi.py
│   ├── comparar_predicciones.py
│   ├── diagnostico_alarmas.py
│   ├── generar_mapa_municipios.py
│   └── verificar_salidas.py
├── src/
│   ├── dashboard/  # Streamlit (solo lee Gold)
│   │   ├── assets/
│   │   │   └── municipios_galicia.geojson
│   │   ├── app.py
│   │   ├── datos.py
│   │   └── graficos.py
│   ├── explainability/  # SHAP
```

```
│   │   ├── config.py
│   │   ├── plots.py
│   │   └── shap_explain.py
│   ├── features/  # Gold: derivadas y dataset municipio-día
│   │   ├── __main__.py
│   │   ├── dataset.py
│   │   └── derived.py
│   ├── ingest/  # Bronze: descarga de fuentes y archivo en HF
│   │   ├── __main__.py
│   │   ├── aemet.py
│   │   ├── cli.py
│   │   ├── common.py
│   │   ├── config.py
│   │   ├── effis.py
│   │   ├── egif.py
│   │   ├── hf_hub.py
│   │   └── ine_cnig.py
│   ├── ml/  # entrenamiento, evaluación, inferencia, publicación
│   │   ├── config.py
│   │   ├── evaluate.py
│   │   ├── experimento.py
│   │   ├── hf_hub.py
│   │   ├── inferir.py
│   │   ├── modelos_hf.py
│   │   ├── predict.py
│   │   ├── reservados.py
│   │   ├── risk.py
│   │   ├── splits.py
│   │   ├── train.py
│   │   └── walkforward.py
│   └── silver/  # Silver: limpieza, tipado y cruce
│       ├── __main__.py
│       ├── almacen.py
│       ├── calidad.py
│       ├── fuentes.py
│       ├── fwi.py
│       ├── geo.py
│       ├── hf_hub.py
│       ├── incendios.py
│       ├── meteo.py
│       ├── muestra_validacion.py
│       ├── municipios.py
│       ├── pipeline.py
│       └── validacion.py
├── tests/  # pytest (datos de juguete, sin red) · 36 ficheros
│       conftest.py, silver_fixtures.py, test_aemet.py, test_cli.py, test_common.py,
│       test_config.py, test_dashboard.py, test_effis.py, test_egif.py,
│       test_errores_red.py, test_evaluate.py, test_experimento.py, test_features.py,
│       test_hf_gold.py, test_hf_hub.py, test_inferir.py, test_modelos_hf.py,
│       test_predict.py, test_reservados.py, test_risk.py, test_run_dashboard.py,
│       test_run_pipeline_ml.py, test_scripts.py, test_shap.py, test_shap_valores.py,
│       test_silver_basicos.py, test_silver_fwi.py, test_silver_hf_hub.py,
│       test_silver_incendios.py, test_silver_meteo.py, test_silver_pipeline.py,
│       test_silver_validacion.py, test_splits.py, test_train.py, test_walkforward.py,
│       toy_data.py
├── data/bronze/ · data/silver/ · data/gold/  # no versionados (solo .gitkeep): Hugging Face
└── models/  # no versionado (solo .gitkeep): modelos v1 en Hugging Face
```

7. Repositorio versionado y reproducible

Evidencia mínima exigida	Dónde está
Enlace accesible al repositorio y tag o commit de entrega	https://github.com/cristinapuertolas-80/iFP-Piriscope · tag `nf3-final` · commit `5e20acbbaf3706ec17a7aa8fc8f1f22de06abbdd`
README con requisitos, configuración y comandos	`README.md`: requisitos (1), instalación (2), credenciales (3), ingesta (4), Silver (5), Gold y ML (6), reproducción completa (6 bis), dashboard (7), tests (8), limitaciones, estructura
Fichero de dependencias o lockfile	`environment/requirements-lock.txt` (exacto), `requirements.txt` (rangos), `requirements-dashboard.txt`
Variables de entorno de ejemplo sin secretos	`.env.example`; `.env` excluido en `.gitignore`; los tokens de CI están en GitHub Secrets
Pruebas automatizadas y evidencias de su resultado	`tests/` (358 pruebas), `tests.yml` con informe JUnit; `evidencias/01_pytest.txt` y `01_pytest.xml`
Estructura real y explicación de módulos	Apartados 6 y 8 de este documento; README de cada carpeta de `src/`
Datos y artefactos versionados o referenciados	Datos y modelos en Hugging Face (`raw/` con un `<timestamp>` por ingesta, `data/silver`, `data/gold`, `models/v1_*`); en GitHub solo ficheros pequeños de referencia: `docs/municipios_reservados.csv` (lista congelada), `docs/tabla_equivalencias.csv`, `docs/resultados/`, GeoJSON del mapa

7.1. Licencias y privacidad de los datos

Todas las fuentes son públicas y reutilizables con atribución: AEMET (Ley 37/2007), EGIF-MITECO (CC BY 4.0 en datos.gob.es), EFFIS/Copernicus (CC BY 4.0), INE y CNIG/IGN (compatible con CC BY 4.0). Las atribuciones están en el README, en cada manifiesto de ingesta y en el pie del dashboard. No se tratan datos personales: los partes se agregan a municipio y día y solo se conservan los campos necesarios. Por eso los datos no se versionan en GitHub, sino en el dataset de Hugging Face con acceso restringido.

8. Explicación por módulo

Raíz del repositorio

- `run_pipeline.py`: orquestador único. Etapas `ingesta`, `silver`, `gold`, `ml` en orden fijo; `--desde/--hasta`, `--fuentes`, `--ml-acciones entrenar inferir`, `--version-modelo`, `--descargar-modelos`, `--inferir-desde/--inferir-hasta`, `--subir-hf` (ingesta, Silver y, tras inferir y verificar, Gold). Se detiene si falla Silver, Gold o ML.
- `run_dashboard.py`: lanzador del dashboard (solo biblioteca estándar): entorno, librerías, token y arranque.
- `.env.example`, `pytest.ini`, `.gitignore` (excluye datos, modelos y secretos), `CONTRIBUTING.md` (flujo Git con ramas y Pull Requests).

src/ingest · Bronze

- `cli.py` y `__main__.py`: `python -m src.ingest <fuente> [--desde --hasta | --dias N] [--subir-hf]`.
- `aemet.py`: climatologías diarias por estación, troceadas en tramos de 6 meses, con caché de los tramos consolidados.
- `egif.py`: unifica las exportaciones Excel de EGIF de `data/landing/egif/` y detecta duplicados.
- `effis.py`: FWI y componentes por mes desde el EWDS (`cdsapi`), consolidado o intermedio, con espera si la cola está llena.
- `ine_cnig.py`: diccionario de municipios del INE y recintos del CNIG.

- `common.py`, `config.py`: manifiestos, reintentos, errores de red, rutas y atribuciones. `hf_hub.py`: subida y descarga de `raw/`.

src/silver · Silver

- `pipeline.py` y `__main__.py`: `python -m src.silver [--descargar-hf] [--desde --hasta] [--tablas]`.
- `municipios.py`, `meteo.py`, `fwi.py`, `incendios.py`: una tabla por entidad del modelo de datos (municipio, estación y observación meteorológica, indicador FWI, evento de incendio).
- `geo.py`, `fuentes.py`, `almacen.py`: utilidades geoespaciales, lectura de Bronze y escritura de Parquet.
- `calidad.py`, `validacion.py`, `muestra_validacion.py`: informes de calidad, validación del modelo de datos (esquema, claves, rangos) y muestra de 50 partes para revisión manual.
- `hf_hub.py`: descarga de `raw/` y subida de Silver.

src/features · Gold

- `derived.py`: `racha_dias_secos`, `precip_acum_7d` y `mes`, usando solo datos ≤ D.
- `dataset.py` y `__main__.py`: une Silver, añade la etiqueta `incendio_d1`, `particion` y `reservado`, y escribe `dataset_municipio_dia.parquet`, `dataset_entrenamiento.parquet` y `gold_informe.json`.

src/ml · Machine Learning

- `config.py`: contrato de entrada (columnas del Anexo A), grupos de variables, variables prohibidas y rutas.
- `splits.py`, `reservados.py`: particiones, escenarios A/B/C y selección reproducible de municipios reservados.
- `train.py`, `evaluate.py`, `risk.py`: entrenamiento (XGBoost/RF), métricas con bootstrap por bloques y umbrales de niveles.
- `experimento.py`, `walkforward.py`: informes de desarrollo, final y walk-forward.
- `predict.py`, `inferir.py`: inferencia batch D+1 con dominio del modelo, respaldo, SHAP y escritura atómica de Gold.
- `modelos_hf.py`: descarga y publicación (en un único commit) de los modelos versionados en Hugging Face.
- `hf_hub.py`: publica `predicciones.parquet` y `explicaciones.parquet` en `data/gold/` del dataset en un único commit, tras comprobar columnas, unicidad y correspondencia entre ambos ficheros.

src/explainability · SHAP

- `shap_explain.py`: top-5 variables por predicción con su valor, valor base y resto; importancia global.
- `plots.py`: gráficos Plotly local, cascada y global (los reutiliza el dashboard). `config.py`: etiquetas y grupos.

src/dashboard · Frontend

- `app.py`: interfaz Streamlit (barra lateral, cuatro pestañas, textos obligatorios).
- `datos.py`: localización y lectura de Gold (HF o local), red de seguridad de duplicados, casos límite, formatos; sin dependencias de Streamlit (se prueba aparte).
- `graficos.py`: mapa coroplético con puntos clicables, evolución del índice y gráficos de evaluación.
- `assets/municipios_galicia.geojson`: límites simplificados de los 313 municipios (catálogo del buscador y del mapa).

scripts · Utilidades

- `carga_inicial.sh` / `.ps1`: carga histórica inicial. `verificar_salidas.py`: comprobación de Gold tras inferir.

- `comparar_predicciones.py`: reproducibilidad frente a las predicciones publicadas. `capturar_evidencias.py`: capturas del dashboard con Playwright.
- `analisis_walkforward.py`, `analisis_niveles.py`, `diagnostico_alarmas.py`, `comparar_fwi.py`: análisis de niveles y del FWI. `generar_mapa_municipios.py`: regenera el GeoJSON.

tests, environment, docs y .github

- `tests/`: un fichero por módulo, con `conftest.py`, `toy_data.py` y `silver_fixtures.py` (datos de juguete con el esquema real).
- `environment/`: dependencias (rangos, lockfile y dashboard).
- `docs/`: entregas NF1 y NF2, arquitectura (diseñada e implementada), `decisiones.md` (DEC-01 a DEC-54), contrato ML ↔ dashboard, plan de ingesta, resultados de evaluación.
- `.github/workflows/`: `tests.yml`, `ingesta.yml`, `evidencias.yml`.

8.1. Fragmentos de código relevantes

Fragmentos literales del commit evaluable que muestran decisiones clave.

Orquestación. `run_pipeline.py` ejecuta las etapas siempre en el orden del pipeline y se detiene si falla una etapa bloqueante (Silver, Gold o ML):

run_pipeline.py, líneas 254–271

```
    ok = True
    for etapa in etapas:
        if etapa == "ingesta":
            exito = etapa_ingesta(args.desde, args.hasta, args.fuentes, args.subir_hf)
        elif etapa == "silver":
            exito = etapa_silver(args.desde, args.hasta, args.subir_hf)
        elif etapa == "gold":
            exito = etapa_gold()
        else:
            exito = etapa_ml(args.ml_acciones, args.version_modelo, args.modelo, args.modelo_respaldo,
                             args.sobrescribir_modelo, args.inferir_desde, args.inferir_hasta,
                             args.reemplazar_predicciones, args.ejemplo_dir, not args.sin_verificar,
                             subir_hf=args.subir_hf, descargar_modelos=args.descargar_modelos)
        ok &= exito
        if not exito and etapa in BLOQUEANTES and not args.continuar:
            log.error("La etapa %s ha fallado: se detiene el pipeline (usa --continuar para seguir)",
etapa)
            break
    return 0 if ok else 1
```

Variables derivadas sin fuga temporal. Gold exige una serie diaria sin huecos por municipio y calcula las derivadas solo con datos hasta D (un test altera los días posteriores y comprueba que no cambian):

src/features/derived.py, líneas 68–75

```
def anadir_derivadas(df: pd.DataFrame, umbral_seco_mm: float = 1.0, min_dias_acum: int = 7) ->
pd.DataFrame:
    """Devuelve una copia con `racha_dias_secos`, `precip_acum_7d` y `mes`."""
    comprobar_serie_diaria(df)
    out = df.copy()
    out["racha_dias_secos"] = racha_dias_secos(out, umbral_seco_mm)
    out["precip_acum_7d"] = precip_acum_7d(out, 7, min_dias_acum).astype("float64")
    out["mes"] = out[COL_FECHA].dt.month.astype("int8")
    return out
```

Dominio del modelo (DEC-48). Las filas en las que falta una variable que nunca faltó en el entrenamiento (el FWI de los últimos días) no se predicen con el modelo principal: se envían al modelo de respaldo sin FWI:

src/ml/inferir.py, líneas 93–99

```
def separar_por_dominio(todo: pd.DataFrame, bundle, desde: pd.Timestamp, hasta: pd.Timestamp):
    """Filas del rango [desde, hasta] dentro y fuera del dominio del modelo."""
    f = pd.to_datetime(todo[config.COL_FECHA]).dt.normalize()
    sub = todo[(f >= desde) & (f <= hasta)]
    dominio = features_sin_nulos(bundle, todo)
    fuera = sub[dominio].isna().any(axis=1) if dominio else pd.Series(False, index=sub.index)
    return sub[~fuera], sub[fuera]
```

Verificación de las salidas (corregido en NF3). Antes, si el script no existía la etapa devolvía éxito y la verificación se omitía en silencio:

run_pipeline.py, líneas 134–149

```
#: Script de verificación de las salidas de Gold (se movió de `src/ml/` a `scripts/`).
SCRIPT_VERIFICAR = Path(__file__).resolve().parent / "scripts" / "verificar_salidas.py"


def verificar_salidas(principal: str, respaldo: str) -> bool:
    """Ejecuta `scripts/verificar_salidas.py` sobre las salidas de Gold.

    Si el script no existe, la etapa falla: omitir la verificación en silencio dejaría pasar salidas sin
comprobar
    (se puede desactivar a propósito con `--sin-verificar`).
    """
    from src.ml import config

    script = SCRIPT_VERIFICAR
    if not script.exists():
        log.error("No existe %s; no se pueden verificar las salidas (usa --sin-verificar para omitirlo)",
script)
        return False
```

Publicación tras verificar (PR #45). Con `--subir-hf`, la etapa ML solo publica la Gold en Hugging Face si la inferencia y la verificación han ido bien:

run_pipeline.py, líneas 203–213

```
        if verificar and not verificar_salidas(modelo, modelo_respaldo):
            log.error("La verificación de las salidas ha encontrado problemas")
            return False
        if subir_hf:
            log.info("=== ML: publicación de las predicciones en Hugging Face ===")
            try:
                url = hf_hub.subir_gold()
            except Exception as e:  # noqa: BLE001 - se informa y la etapa devuelve error
                log.error("No se pudo publicar Gold en Hugging Face: %s", e)
                return False
            log.info("Gold publicado: %s", url)
```

El dashboard solo lee Gold. Por defecto la descarga de Hugging Face; con `--local` (o sin conexión) usa `data/gold/`:

src/dashboard/datos.py, líneas 74–83

```
def localizar_gold(forzar_descarga: bool = False, gold_dir: Path | None = None) -> OrigenGold:
    """Rutas de los ficheros de Gold: Hugging Face por defecto, ``data/gold`` como respaldo."""
    gold_dir = Path(gold_dir or config.GOLD_DIR)
    locales = {n: gold_dir / n for n in FICHEROS_GOLD}
    hay_locales = all(p.exists() for p in locales.values())

    if os.getenv("PIRISCOPE_DASHBOARD_ORIGEN", "hf").strip().lower() == "local":
        if not hay_locales:
            raise FileNotFoundError(f"No están {', '.join(FICHEROS_GOLD)} en {gold_dir}")
        return OrigenGold(locales["predicciones.parquet"], locales["explicaciones.parquet"], "local")
```

9. Limitaciones actuales

9.1. Bugs

Encontrados y corregidos en esta entrega (PR #44 con test de regresión, y PR #47):

- `run_pipeline.py` buscaba `src/ml/verificar_salidas.py`, que se había movido a `scripts/`, y devolvía éxito si no lo encontraba: la etapa ML **omitía en silencio** la verificación de las salidas. Ahora la ejecuta y falla si falta.
- `scripts/analisis_niveles.py` no se podía ejecutar tras moverlo (imports relativos). Un test comprueba ahora que todos los scripts arrancan.
- El README daba un AUC-PR de 0,094 en el escenario B; el informe final dice 0,0935 (0,093).
- Tras la PR #45, dos decisiones compartían el número DEC-53 y varios ficheros citaban DEC-52 para la publicación de Gold; la PR #47 renumera la de la entrega NF3 como DEC-54 y alinea las referencias.

Conocidos, sin corregir:

- En la tabla del día, las celdas vacías (FWI de las estimaciones sin FWI, municipios sin dato) se muestran como «None», que es como `st.dataframe` pinta los nulos (figura 4). Convertirlas a texto rompería la ordenación numérica.
- En la explicación SHAP, el título del eje horizontal se corta en algunos anchos de ventana (figura 3).

9.2. Atajos

- La publicación de Gold y modelos en Hugging Face (DEC-53) solo se ha probado con un cliente simulado y con `--dry-run`; la Gold publicada hoy (hasta el 22/10/2026) se subió antes de existir ese comando.
- Las exportaciones de EGIF y los recintos del CNIG se descargan a mano a `data/landing/`.
- El viento se excluyó del modelo (57 % de nulos) y el resto de variables meteorológicas se dejan con nulos, sin imputar (18-27 %).
- Los niveles de riesgo usan umbrales fijados con un solo año de validación (2022) y la probabilidad no está calibrada: se muestra como «índice de riesgo».
- Ejecutar requiere acceso al dataset restringido de Hugging Face; no hay una muestra pública pequeña para probar sin cuenta.

9.3. Partes incompletas

- Actualización diaria automática de las predicciones (hoy: ingesta automática; Silver, Gold, inferencia y publicación se lanzan a mano).
- Cobertura de los días recientes: entre 286 y 288 de los 313 municipios en las últimas fechas (mínimo 91 %). Supera el 80 % exigido, pero los municipios sin ninguna variable meteorológica salen como «Sin dato».
- Reentrenamiento reproducible no verificado con datos reales; política de niveles (DEC-49) y comparación FWI intermedio/consolidado pendientes.

9.4. Limitaciones del modelo

- Mejora modesta frente al FWI: en el test de 2023 (municipios vistos) AUC-PR 0,028 frente a 0,019 (diferencia +0,008, IC 95 % +0,001 a +0,019); un único año de test.
- A Coruña no mejora al FWI; el modelo de respaldo (sin FWI) equivale al FWI.
- Precision del 2 % al 6 % con Recall ≈ 0,8: «medio» o «alto» significa «no descartar», no «habrá un incendio».

10. Lista de comprobación de reproducibilidad

Comprobación	Estado	Evidencia
Repositorio accesible	✔️	Público: https://github.com/cristinapuertolas-80/iFP-Piriscope
Tag o commit exacto e inmutable	✔️	`nf3-final` → `5e20acbbaf37`
README actualizado	✔️	Estado NF3, guía 6 bis, estructura real, limitaciones
Dependencias con versiones exactas	✔️	`environment/requirements-lock.txt` (instalado en CI)
Variables de entorno de ejemplo sin secretos	✔️	`.env.example`
Tests automatizados en verde	✔️	357 pasan, 1 omitida en el commit evaluable
Ejecución desde cero en máquina limpia	✔️	Ejecución 42391224431: Silver → Gold → ML → dashboard
Salidas verificadas automáticamente	✔️	`verificar_salidas.py`: «Todo correcto»
Predicciones reproducibles	✔️	Con las mismas entradas, mismas salidas (diferencia ≤ $1{,}9 \cdot 10^{-9}$)
Datos y modelos referenciados con licencia	✔️	Hugging Face (`raw/`, `data/`, `models/`); licencias en README
Documentación correspondiente al commit	✔️	Este documento se generó a partir del commit evaluable y de su ejecución de evidencias

11. Condiciones de la entrega

- La documentación corresponde al tag `nf3-final` (commit `5e20acbbaf3706ec17a7aa8fc8f1f22de06abbdd`).
- No se modificará el código de ese tag tras la entrega. Cualquier cambio posterior irá en commits nuevos y no afecta a la puntuación de esta fase salvo autorización.
- El tag o commit y esta documentación forman la referencia oficial de corrección.

Anexo A. Contribuciones individuales

Todo el código llegó a `main` mediante Pull Requests (`CONTRIBUTING.md`). Contribuciones por Pull Request fusionado:

Integrante (usuario de GitHub)	Pull Requests	Contenido
Cristina Puértolas (cristinapuertolas-80)	#31, #32, #33	Ingesta de las cuatro fuentes en Bronze, plan de ingesta, carga inicial, workflow de ingesta, errores de red
Guillermo Ramón (Gramser-DE)	#34	Capa Silver: limpieza, tipado, cruce espacio-temporal, calidad y validación
Hikari Lheku (MoonPresence)	#5, #7, #8, #15, #16, #30, #35–#41, #45, #46	Estructura inicial del repositorio, Gold, ML (entrenamiento, evaluación, walk-forward, reservados, inferencia), SHAP, resultados, publicación de Gold y modelos en Hugging Face
Alejandro Escolar (alexescolar88-code)	#42, #43	Dashboard Streamlit y lanzador en un comando
Equipo (preparación NF3)	#44, #47	Correcciones, lockfile, descarga de modelos, workflow de evidencias, alineación de la documentación, documento de la entrega

El registro completo de decisiones, con su responsable, está en `docs/decisiones.md`.